

CI/CD Project Setup Guide — GitHub + Jenkins + SonarQube + Docker (AWS)

A complete beginner-friendly walkthrough. Follow the steps in order — don't skip ahead.

Overview of what you're building

1. You push Java code to GitHub.
2. GitHub notifies Jenkins automatically (webhook).
3. Jenkins builds the code, runs a SonarQube quality scan, builds a Docker image, and runs the container.
4. Your app is now live on your EC2 server.

Everything (Jenkins, SonarQube, Docker) will run on **one AWS EC2 instance** to keep this simple.

Part 1 — Launch your AWS EC2 instance

1. Log in to the [AWS Console](#).
2. Go to **EC2** → **Instances** → **Launch Instance**.
3. Fill in:
 - **Name:** `cicd-server`
 - **AMI (OS):** Ubuntu Server 22.04 LTS
 - **Instance type:** `t2.medium` (minimum — Jenkins + SonarQube + Docker together need at least 4GB RAM. A `t2.micro` will struggle badly.)
 - **Key pair:** Create a new key pair, name it (e.g. `cicd-key`), download the `.pem` file, and keep it safe — you need it to SSH in.
 - **Network settings** → click **Edit** and add these inbound rules (in addition to the default SSH rule):

Type	Port	Source
SSH	22	My IP
Custom TCP	8080	Anywhere (0.0.0.0/0) — Jenkins
Custom TCP	9000	Anywhere (0.0.0.0/0) — SonarQube
Custom TCP	8081	Anywhere (0.0.0.0/0) — your app (we'll use 8081 to avoid clashing with SonarQube's 9000/Jenkins's 8080)

- **Storage:** bump to at least 20 GB (default 8GB is too small once you add Jenkins + SonarQube + Docker images).

4. Click **Launch Instance**.

5. Once it's running, copy its **Public IPv4 address** — you'll use this a lot.

Part 2 — Connect to your server

On your local machine (Mac/Linux terminal, or Git Bash / WSL on Windows):

```
bash

chmod 400 cisd-key.pem
ssh -i cisd-key.pem ubuntu@<YOUR_EC2_PUBLIC_IP>
```

If it connects, you're in. Everything from here happens **on the server**, unless stated otherwise.

Part 3 — Install Java, Jenkins, and Docker

Run these one at a time so you can see if anything fails:

```
bash

# Update package list
sudo apt update && sudo apt upgrade -y

# Install Java (Jenkins needs it)
sudo apt install -y openjdk-17-jdk
java -version    # should print a version — confirms it worked
```

Install Jenkins:

```
bash

sudo wget -O /usr/share/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/deb
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenki
sudo apt update
sudo apt install -y jenkins
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins    # should show "active (running)" — press q to e
```

Install Docker:

```
bash
```

```
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker

# Let Jenkins run docker commands
sudo usermod -aG docker jenkins
sudo usermod -aG docker ubuntu
sudo systemctl restart jenkins
```

Install Maven (needed to build your Java project):

```
bash
```

```
sudo apt install -y maven
mvn -version # confirms it worked
```

Part 4 — Open Jenkins in your browser and unlock it

1. In your browser, go to: `http://<YOUR_EC2_PUBLIC_IP>:8080`
2. It will ask for an **Administrator password**. Get it by running this on the server:

```
bash
```

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

3. Copy that password, paste it into the browser, click **Continue**.
4. Click **Install suggested plugins** — wait for it to finish (a few minutes).
5. Create your first admin user (username/password you'll remember) → **Save and Continue** → **Save and Finish** → **Start using Jenkins**.

You now have Jenkins running.

Part 5 — Install extra Jenkins plugins

You need a few plugins beyond the defaults:

1. In Jenkins, go to **Manage Jenkins** → **Plugins** → **Available plugins** tab.
2. Search for and install each of these (check the box, then click **Install**):
 - **GitHub Integration Plugin** (if not already present)
 - **SonarQube Scanner**
 - **Pipeline: Maven Integration** (or just **Maven Integration**)

- **Docker Pipeline**

3. Tick "**Restart Jenkins when installation is complete**" and let it restart. Log back in afterward.

Part 6 — Set up SonarQube

Easiest way: run it as a Docker container on the same server.

```
bash
```

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:community
```

Wait about a minute for it to start, then:

1. Open `http://<YOUR_EC2_PUBLIC_IP>:9000` in your browser.
2. Log in with default credentials: **admin** / **admin**. It will ask you to set a new password immediately — do that.
3. Once logged in, go to **My Account** (top right) → **Security** → under **Generate Tokens**, type a name like `jenkins-token` → **Generate**.
4. **Copy the token immediately** — it's shown only once. Paste it somewhere safe temporarily (you'll need it in Part 7).

Part 7 — Connect Jenkins to SonarQube

1. In Jenkins: **Manage Jenkins** → **Credentials** → click (global) → **Add Credentials**.
 - Kind: **Secret text**
 - Secret: paste the SonarQube token from Part 6
 - ID: `sonarqube-token`
 - Click **Create**.
2. **Manage Jenkins** → **System** → scroll to **SonarQube servers** → check **Enable injection of SonarQube server configuration**.
 - Click **Add SonarQube** →
 - Name: `SonarQube`
 - Server URL: `http://localhost:9000`
 - Server authentication token: select the `sonarqube-token` credential you just made
 - **Save**.
3. **Manage Jenkins** → **Tools** → scroll to **SonarQube Scanner installations** → **Add SonarQube Scanner** → name it `SonarScanner` → let it auto-install → **Save**.
4. Still in **Tools**, also configure:

- **Maven installations** → Add Maven → name it `Maven3` → check "Install automatically" → **Save**.
- **JDK installations** → Add JDK → name it `JDK17` → point it to the system JDK path (`/usr/lib/jvm/java-17-openjdk-amd64`) or let it auto-install → **Save**.

These names (`SonarQube`, `SonarScanner`, `Maven3`, `JDK17`) must match exactly what's used in your Jenkinsfile.

Part 8 — Push your Java project to GitHub

If you haven't already:

1. Create a new repository on [GitHub](#) (e.g. `my-cicd-app`).
2. On your local machine, inside your Java project folder:

bash

```
git init
git add .
git commit -m "initial commit"
git branch -M main
git remote add origin https://github.com/<your-username>/my-cicd-app.git
git push -u origin main
```

3. Add two files to the **root** of your project (next to `pom.xml`):

Dockerfile:

dockerfile

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Jenkinsfile:

groovy

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "myapp"
        CONTAINER_NAME = "myapp-container"
        APP_PORT = "8081"
    }

    tools {
        maven 'Maven3'
        jdk 'JDK17'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-username>/my
            }
        }

        stage('Build & Test') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh 'mvn sonar:sonar -Dsonar.projectKey=myapp'
                }
            }
        }

        stage('Quality Gate') {
            steps {
                timeout(time: 2, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }
    }
}
```

```

    stage('Build Docker Image') {
        steps {
            sh 'docker build -t $IMAGE_NAME:latest .'
        }
    }

    stage('Deploy Container') {
        steps {
            sh '''
                docker stop $CONTAINER_NAME || true
                docker rm $CONTAINER_NAME || true
                docker run -d --name $CONTAINER_NAME -p $APP_PORT:8080 $IMA
            '''
        }
    }
}

```

Commit and push both files:

```

bash

git add Dockerfile Jenkinsfile
git commit -m "add Dockerfile and Jenkinsfile"
git push

```

Part 9 — Create the Jenkins pipeline job

1. In Jenkins, click **New Item**.
2. Name it (e.g. `my-cicd-pipeline`), select **Pipeline**, click **OK**.
3. Scroll to **Build Triggers** → check **GitHub hook trigger for GITScm polling**.
4. Scroll to **Pipeline** section:
 - Definition: **Pipeline script from SCM**
 - SCM: **Git**
 - Repository URL: `https://github.com/<your-username>/my-cicd-app.git`
 - Branch: `*/main`
 - Script Path: `Jenkinsfile` (default, leave as is)
5. Click **Save**.

Part 10 — Connect GitHub to Jenkins (the webhook)

1. Go to your GitHub repo → **Settings** → **Webhooks** → **Add webhook**.

2. Fill in:

- **Payload URL:** `http://<YOUR_EC2_PUBLIC_IP>:8080/github-webhook/`
- **Content type:** `application/json`
- **Which events:** "Just the push event"

3. Click **Add webhook**.

4. GitHub will send a test ping — check under the webhook's **Recent Deliveries** tab for a green checkmark confirming Jenkins received it.

Part 11 — Run it end to end

1. Make a small change to your code locally, commit, and push:

```
bash
```

```
git add .
git commit -m "test trigger"
git push
```

2. Within a few seconds, go to Jenkins → your pipeline job — a new build should start automatically.
 3. Click on the build number → **Console Output** to watch it progress through: checkout → build → SonarQube scan → quality gate → Docker build → deploy.
 4. Once it finishes successfully, open `http://<YOUR_EC2_PUBLIC_IP>:8081` in your browser — your app should be live.
 5. Check SonarQube at `http://<YOUR_EC2_PUBLIC_IP>:9000` → your project should appear with a quality report.
-

Troubleshooting tips

- **Webhook not triggering Jenkins:** check the security group allows inbound traffic on 8080 from anywhere, and check "Recent Deliveries" in GitHub for the error response.
 - **`docker: permission denied`:** make sure you ran `sudo usermod -aG docker jenkins` and restarted Jenkins (`sudo systemctl restart jenkins`).
 - **SonarQube analysis fails / times out:** your EC2 instance may be too small — SonarQube alone needs ~2GB RAM. Upgrade to `t2.medium` or larger if builds hang.
 - **Build fails at "Quality Gate" stage:** this is expected if your code has serious code-quality issues flagged by SonarQube's default rules — check the SonarQube dashboard for details.
 - **Port already in use:** if 8081 clashes with something, just change `APP_PORT` in the Jenkinsfile and open that port in your security group instead.
-

Recap of what's running where

Service	Port	URL
Jenkins	8080	<code>http://<EC2_IP>:8080</code>
SonarQube	9000	<code>http://<EC2_IP>:9000</code>
Your app	8081	<code>http://<EC2_IP>:8081</code>

That's the full loop, start to finish. Once this works, natural next steps (only if you want them later) are adding automated tests, a staging environment, or moving the image to a registry like Docker Hub or AWS ECR — but none of that is required for what you asked for.